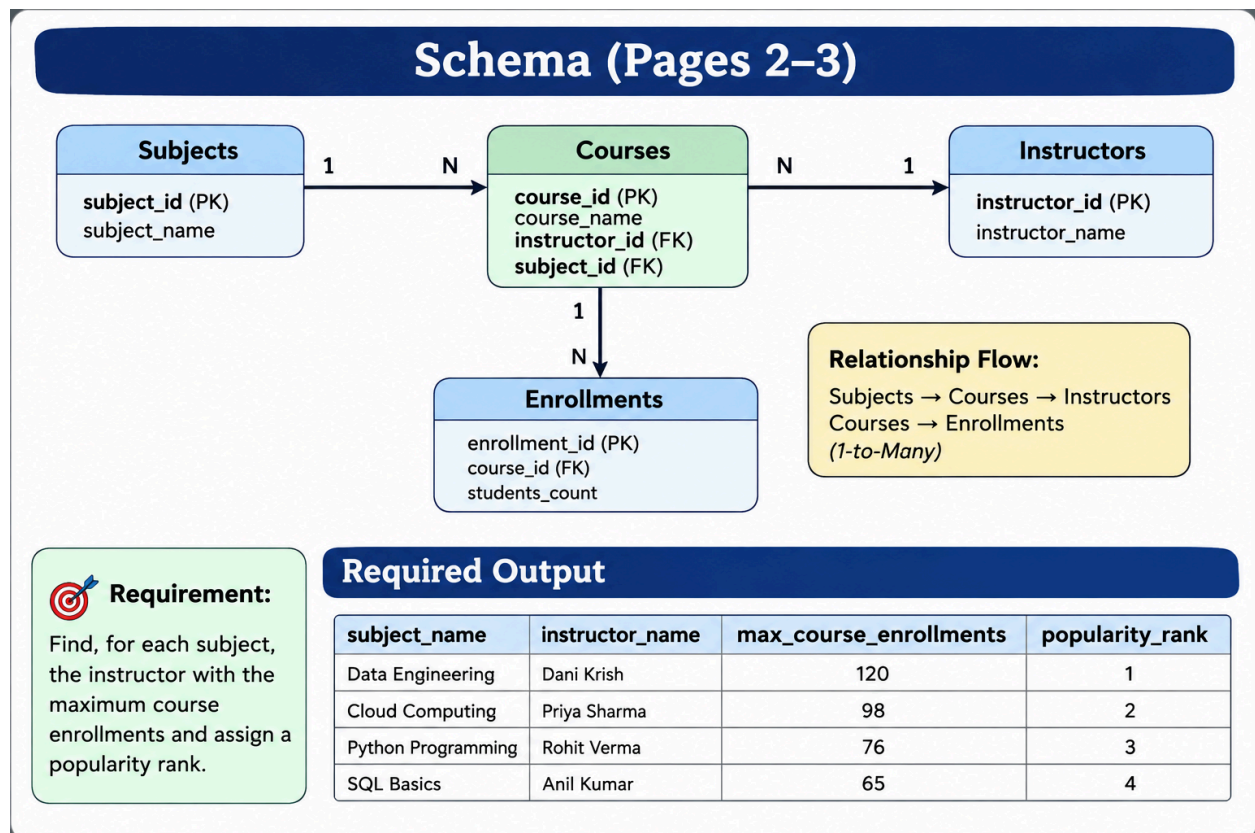


COD-Week-2-Day-1

The problem explicitly defines the popularity measure as the highest enrollment achieved by any one course taught by the instructor.

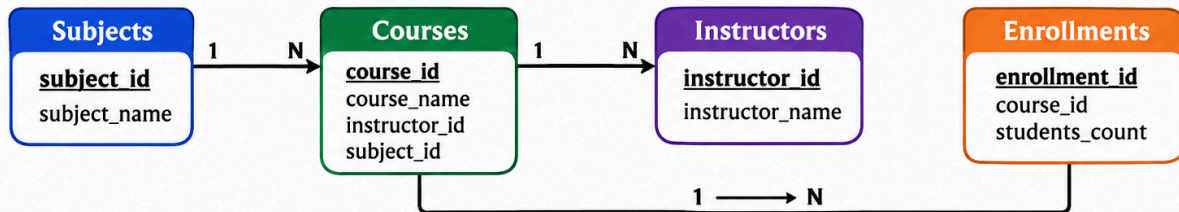
1. First understand the exact problem

The four tables are:



Most Popular Instructor per Subject

Find the instructor with the highest course enrollments for each subject and rank the instructors by popularity.



Required Output:

• subject_name | instructor_name | max_course_enrollments | popularity_rank

subject_name	instructor_name	max_course_enrollments	popularity_rank
Data Engineering	Dani YK	1250	1
Python Programming	Ravi Sharma	980	2
Cloud Computing	Priya Menon	870	3
SQL & Databases	Arjun Rao	760	4
Big Data & Spark	Sneha Iyer	690	5

2. The most important insight: determine the grain

Before writing SQL, ask:

What does one row represent?

According to the **SQL problem solving framework** explicitly recommends identifying the output grain before selecting the SQL technique.

For this problem:

Intermediate grain #1

ONE ROW = ONE COURSE

Because `students_count` belongs to a course.

Example:

Subject	Instructor	Course	Students
Data Science	John Doe	Python for DS	500
Data Science	Sarah Lee	ML Basics	450
Data Science	Adam White	Deep Learning	450

Intermediate grain #2

After MAX():

ONE ROW = ONE INSTRUCTOR + ONE SUBJECT

Subject	Instructor	Max Course Enrollments
Data Science	John Doe	500
Data Science	Sarah Lee	450
Data Science	Adam White	450

Final grain

Still:

ONE ROW = ONE INSTRUCTOR + ONE SUBJECT

but with a rank added.

3. Why SUM() is WRONG here

This is the biggest trap in the question.

Suppose John teaches:

Python for DS 500 students
Deep Learning 300 students
Machine Learning 200 students

The question does **not** ask:

$$500 + 300 + 200 = 1000$$

It asks:

$$\text{MAX}(500, 300, 200) = 500$$

The problem explicitly says the popularity is based on the **highest number of students enrolled in any one course** taught by the instructor.

Therefore:

MAX(students_count)

is required.

4. Method 1 — GROUP BY + MAX() + RANK()

This is the solution that most directly matches the problem title.

Layer 1 — Join the four tables

```
SELECT
    s.subject_name,
    i.instructor_name,
    c.course_id,
    c.course_name,
    e.students_count
FROM subjects s
JOIN courses c
    ON s.subject_id = c.subject_id
JOIN instructors i
    ON c.instructor_id = i.instructor_id
JOIN enrollments e
    ON c.course_id = e.course_id;
```

This produces course-level data.

Layer 2 — Find the maximum course enrollment per instructor/subject

```
SELECT
    s.subject_name,
    i.instructor_name,
    MAX(e.students_count) AS max_course_enrollments
FROM subjects s
```

```

JOIN courses c
  ON s.subject_id = c.subject_id
JOIN instructors i
  ON c.instructor_id = i.instructor_id
JOIN enrollments e
  ON c.course_id = e.course_id
GROUP BY
  s.subject_id,
  s.subject_name,
  i.instructor_id,
  i.instructor_name;

```

Now we have:

```

Data Science | John Doe   | 500
Data Science | Sarah Lee  | 450
Data Science | Adam White | 450
Web Dev      | Alice Green | 300
Web Dev      | David Park  | 200

```

Layer 3 — Rank instructors within subject

```

WITH instructor_popularity AS
(
  SELECT
    s.subject_id,
    s.subject_name,
    i.instructor_id,
    i.instructor_name,
    MAX(e.students_count) AS max_course_enrollments
  FROM subjects s
  JOIN courses c
    ON s.subject_id = c.subject_id
  JOIN instructors i
    ON c.instructor_id = i.instructor_id
  JOIN enrollments e
    ON c.course_id = e.course_id
  GROUP BY
    s.subject_id,
    s.subject_name,
    i.instructor_id,
    i.instructor_name
)

```

```
SELECT
    subject_name,
    instructor_name,
    max_course_enrollments,
    RANK() OVER
    (
        PARTITION BY subject_id
        ORDER BY max_course_enrollments DESC
    ) AS popularity_rank
FROM instructor_popularity
ORDER BY
    subject_name,
    popularity_rank;
```

This is the cleanest answer.

It follows exactly:

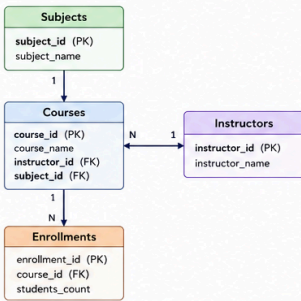
```
JOIN
↓
MAX()
↓
GROUP BY instructor + subject
↓
RANK()
↓
PARTITION BY subject
```

The assignment explicitly requires **RANK()** and equal values must receive the same rank.

Method 4 — Derived Table instead of CTE

Goal: For each subject, find the instructor(s) with the maximum course enrollments and rank instructors by that maximum (popularity rank).

Database Schema



Sample Data

Subjects		Instructors	
subject_id	subject_name	instructor_id	instructor_name
1	Data Engineering	1	Dani Krish
2	Cloud Computing	2	Priya Sharma
3	Python Programming	3	Rohit Verma
4	SQL Basics	4	Anil Kumar
		5	Amit Singh

Courses		Enrollments		
course_id	course_name	instructor_id	subject_id	students_count
101	DE Basics	1	1	120
102	Big Data	1	1	80
201	Cloud Intro	2	2	98
202	AWS Basics	5	2	98
301	Python Core	3	3	76
401	SQL Fundamentals	4	4	65

SQL Query (Derived Table Approach)

```

SELECT
  x.subject_name,
  x.instructor_name,
  x.max_course_enrollments,
  RANK() OVER
  (
    PARTITION BY x.subject_id
    ORDER BY x.max_course_enrollments DESC
  ) AS popularity_rank
FROM
(
  SELECT
    s.subject_id,
    s.subject_name,
    i.instructor_id,
    i.instructor_name,
    MAX(e.students_count) AS max_course_enrollments
  FROM subjects s
  JOIN courses c ON s.subject_id = c.subject_id
  JOIN instructors i ON c.instructor_id = i.instructor_id
  JOIN enrollments e ON c.course_id = e.course_id
  GROUP BY
    s.subject_id,
    s.subject_name,
    i.instructor_id,
    i.instructor_name
) x
ORDER BY
  x.subject_name,
  popularity_rank;
  
```

Conceptual Flow



Step 1 & 2 Result: Derived Table Output (Before Ranking)

subject_id	subject_name	instructor_id	instructor_name	max_course_enrollments
1	Data Engineering	1	Dani Krish	120
1	Data Engineering	1	Dani Krish	80
2	Cloud Computing	2	Priya Sharma	98
2	Cloud Computing	5	Amit Singh	98
3	Python Programming	3	Rohit Verma	76
4	SQL Basics	4	Anil Kumar	65

Note: MAX(students_count) is taken per (subject, instructor) across all their courses.

Final Output (After RANK)

subject_name	instructor_name	max_course_enrollments	popularity_rank
Cloud Computing	Priya Sharma	98	1
Cloud Computing	Amit Singh	98	1
Data Engineering	Dani Krish	120	1
Python Programming	Rohit Verma	76	1
SQL Basics	Anil Kumar	65	1

✓ Ties get the same rank.
 Ranking is done within each subject (partitioned by subject).

5. Method 2 — MAX() OVER() + RANK()

We can also use a window function for the maximum.

First produce course-level data:

WITH course_data AS

(

```

SELECT
  s.subject_id,
  s.subject_name,
  i.instructor_id,
  i.instructor_name,
  c.course_id,
  e.students_count
FROM subjects s
  
```

```

JOIN courses c
  ON s.subject_id = c.subject_id
JOIN instructors i
  ON c.instructor_id = i.instructor_id
JOIN enrollments e
  ON c.course_id = e.course_id
),
instructor_data AS
(
  SELECT DISTINCT
    subject_id,
    subject_name,
    instructor_id,
    instructor_name,
    MAX(students_count) OVER
      (
        PARTITION BY subject_id, instructor_id
      ) AS max_course_enrollments
  FROM course_data
)
SELECT
  subject_name,
  instructor_name,
  max_course_enrollments,
  RANK() OVER
    (
      PARTITION BY subject_id
      ORDER BY max_course_enrollments DESC
    ) AS popularity_rank
FROM instructor_data
ORDER BY
  subject_name,
  popularity_rank;

```

This is valid, but for teaching this particular problem I prefer Method 1 because `MAX()` naturally expresses the aggregation requirement.

6. Method 3 — Two CTEs

This is probably the **best teaching version** because each business transformation gets its own layer.

WITH course_enrollments AS

```
(
  SELECT
    s.subject_id,
    s.subject_name,
    i.instructor_id,
    i.instructor_name,
    c.course_id,
    e.students_count
  FROM subjects s
  JOIN courses c
    ON s.subject_id = c.subject_id
  JOIN instructors i
    ON c.instructor_id = i.instructor_id
  JOIN enrollments e
    ON c.course_id = e.course_id
),
```

instructor_popularity AS

```
(
  SELECT
    subject_id,
    subject_name,
    instructor_id,
    instructor_name,
    MAX(students_count) AS max_course_enrollments
  FROM course_enrollments
  GROUP BY
    subject_id,
    subject_name,
    instructor_id,
    instructor_name
),
```

ranked_instructors AS

```
(
  SELECT
    subject_name,
    instructor_name,
    max_course_enrollments,
    RANK() OVER
      (
        PARTITION BY subject_id
        ORDER BY max_course_enrollments DESC
      )
  FROM instructor_popularity
)
```

```
    ) AS popularity_rank  
FROM instructor_popularity  
)
```

```
SELECT *  
FROM ranked_instructors  
ORDER BY  
    subject_name,  
    popularity_rank;
```

This matches the problem-solving framework's principle:

Break a multi-stage problem into logical layers.

Your trainee framework explicitly identifies CTEs as named intermediate datasets and recommends them for multiple transformations.

7. Method 4 — Derived table instead of CTE

If you don't want a CTE:

```
SELECT  
    x.subject_name,  
    x.instructor_name,  
    x.max_course_enrollments,  
    RANK() OVER  
    (  
        PARTITION BY x.subject_id  
        ORDER BY x.max_course_enrollments DESC  
    ) AS popularity_rank  
FROM  
    (  
        SELECT  
            s.subject_id,  
            s.subject_name,  
            i.instructor_id,  
            i.instructor_name,  
            MAX(e.students_count) AS max_course_enrollments  
        FROM subjects s
```

```

JOIN courses c
  ON s.subject_id = c.subject_id
JOIN instructors i
  ON c.instructor_id = i.instructor_id
JOIN enrollments e
  ON c.course_id = e.course_id
GROUP BY
  s.subject_id,
  s.subject_name,
  i.instructor_id,
  i.instructor_name
) x
ORDER BY
  x.subject_name,
  popularity_rank;

```

Conceptually:

```

Derived Table
  ↓
Instructor popularity
  ↓
RANK()

```

8. Method 5 — **RANK()** directly after aggregation

You can actually reduce the query further in MySQL 8+:

```

SELECT
  s.subject_name,
  i.instructor_name,
  MAX(e.students_count) AS max_course_enrollments,

  RANK() OVER
  (
    PARTITION BY s.subject_id
    ORDER BY MAX(e.students_count) DESC
  ) AS popularity_rank

FROM subjects s

```

```
JOIN courses c
  ON s.subject_id = c.subject_id

JOIN instructors i
  ON c.instructor_id = i.instructor_id

JOIN enrollments e
  ON c.course_id = e.course_id

GROUP BY
  s.subject_id,
  s.subject_name,
  i.instructor_id,
  i.instructor_name

ORDER BY
  s.subject_name,
  popularity_rank;
```

Why does this work?

SQL logically performs:

```
FROM
↓
JOIN
↓
GROUP BY
↓
MAX()
↓
Window function
↓
ORDER BY
```

So the window function can rank the aggregated result.

This is a very nice demonstration of the combination:

GROUP BY + MAX() + RANK()

9. Method 6 — DENSE_RANK()

Technically possible:

```
DENSE_RANK() OVER  
(  
  PARTITION BY s.subject_id  
  ORDER BY MAX(e.students_count) DESC  
)
```

But **do not use it for this assignment**.

The assignment specifically says:

Use **RANK()**.

For this data:

500

450

450

both produce:

RANK	DENSE_RANK
------	------------

500 → 1	500 → 1
---------	---------

450 → 2	450 → 2
---------	---------

450 → 2	450 → 2
---------	---------

But if there were:

500

450

450

300

then:

RANK:

1

2

2

4

DENSE_RANK:

1
2
2
3

Since the requirement is explicitly `RANK()`, use `RANK()`.

10. Method 7 — `ROW_NUMBER()`

Possible technically:

```
ROW_NUMBER() OVER  
(  
    PARTITION BY s.subject_id  
    ORDER BY MAX(e.students_count) DESC  
)
```

But **wrong for the assignment's tie requirement**.

The sample has:

Sarah Lee 450
Adam White 450

The expected output gives both:

Sarah Lee 450 2
Adam White 450 2

The PDF's expected output confirms this.

`ROW_NUMBER()` would instead produce:

Sarah Lee 450 2
Adam White 450 3

or potentially reverse them depending on the execution plan/order.

So:

Don't use `ROW_NUMBER()` here.

11. Method 8 — **MAX()** + correlated subquery

You can also solve the "maximum per instructor" portion using a correlated subquery.

Conceptually:

```
SELECT
    s.subject_name,
    i.instructor_name,
    e.students_count
FROM subjects s
JOIN courses c
    ON s.subject_id = c.subject_id
JOIN instructors i
    ON c.instructor_id = i.instructor_id
JOIN enrollments e
    ON c.course_id = e.course_id
WHERE e.students_count =
(
    SELECT MAX(e2.students_count)
    FROM courses c2
    JOIN enrollments e2
        ON c2.course_id = e2.course_id
    WHERE c2.instructor_id = c.instructor_id
    AND c2.subject_id = c.subject_id
);
```

This identifies the course(s) representing the instructor's maximum enrollment.

But then you still need ranking.

Therefore it becomes more complicated than necessary.

For this problem:

CTE + GROUP BY + MAX + RANK is cleaner.

12. Method 9 — **MAX()** then self-join

Another possible approach:

WITH instructor_courses AS

```
(
  SELECT
    s.subject_id,
    s.subject_name,
    i.instructor_id,
    i.instructor_name,
    e.students_count
  FROM subjects s
  JOIN courses c
    ON s.subject_id = c.subject_id
  JOIN instructors i
    ON c.instructor_id = i.instructor_id
  JOIN enrollments e
    ON c.course_id = e.course_id
),
```

instructor_max AS

```
(
  SELECT
    subject_id,
    instructor_id,
    MAX(students_count) AS max_course_enrollments
  FROM instructor_courses
  GROUP BY
    subject_id,
    instructor_id
)
```

```
SELECT
  im.subject_id,
  ic.subject_name,
  ic.instructor_name,
  im.max_course_enrollments,
  RANK() OVER
  (
    PARTITION BY im.subject_id
    ORDER BY im.max_course_enrollments DESC
  ) AS popularity_rank
```



```
FROM instructor_max im
JOIN instructor_courses ic
  ON im.subject_id = ic.subject_id
  AND im.instructor_id = ic.instructor_id
GROUP BY
  im.subject_id,
  ic.subject_name,
  ic.instructor_name,
  im.max_course_enrollments
ORDER BY
  ic.subject_name,
  popularity_rank;
```

It works, but there's unnecessary complexity.

13. What the sample data tells us

The sample data on pages 4–5 is particularly useful.

Data Science

Courses:

Python for DS	John Doe	500
ML Basics	Sarah Lee	450
Deep Learning	Adam White	450

Therefore:

John Doe	→ MAX = 500
Sarah Lee	→ MAX = 450
Adam White	→ MAX = 450

Ranking:

John Doe	→ 1
Sarah Lee	→ 2
Adam White	→ 2

Web Dev

HTML & CSS	Alice Green	300
------------	-------------	-----

JavaScript Basics David Park 200

Therefore:

Alice Green $\rightarrow 300 \rightarrow 1$

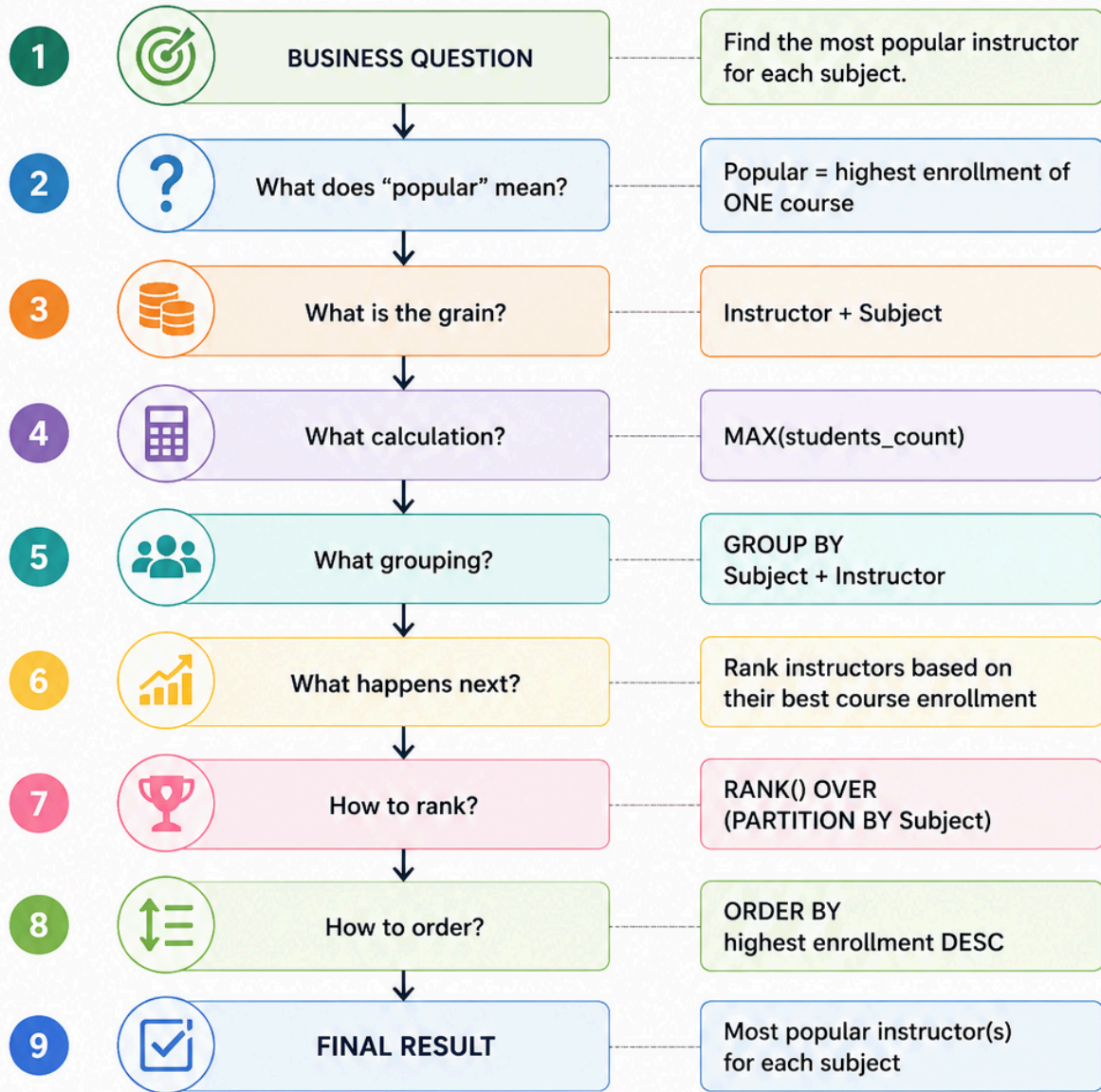
David Park $\rightarrow 200 \rightarrow 2$

Exactly as shown in the expected output.

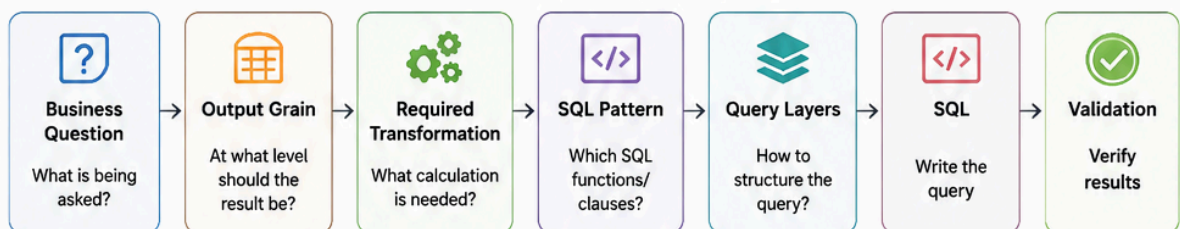
14. Final recommended answer

SQL PROBLEM-SOLVING PATTERN

Most Popular Instructor per Subject



MAPPING TO THE SQL FRAMEWORK



ONE-LINE MENTAL MODEL FOR TRAINEES



"First find each instructor's best course, then rank those **instructor-level bests** within each subject."

This is the query I would submit:

```
SELECT
  s.subject_name,
  i.instructor_name,
  MAX(e.students_count) AS max_course_enrollments,

  RANK() OVER
  (
    PARTITION BY s.subject_id
    ORDER BY MAX(e.students_count) DESC
  ) AS popularity_rank

FROM subjects s

JOIN courses c
  ON s.subject_id = c.subject_id

JOIN instructors i
  ON c.instructor_id = i.instructor_id

JOIN enrollments e
  ON c.course_id = e.course_id

GROUP BY
  s.subject_id,
  s.subject_name,
  i.instructor_id,
  i.instructor_name

ORDER BY
  s.subject_name,
  Popularity_rank;
```

Expected result

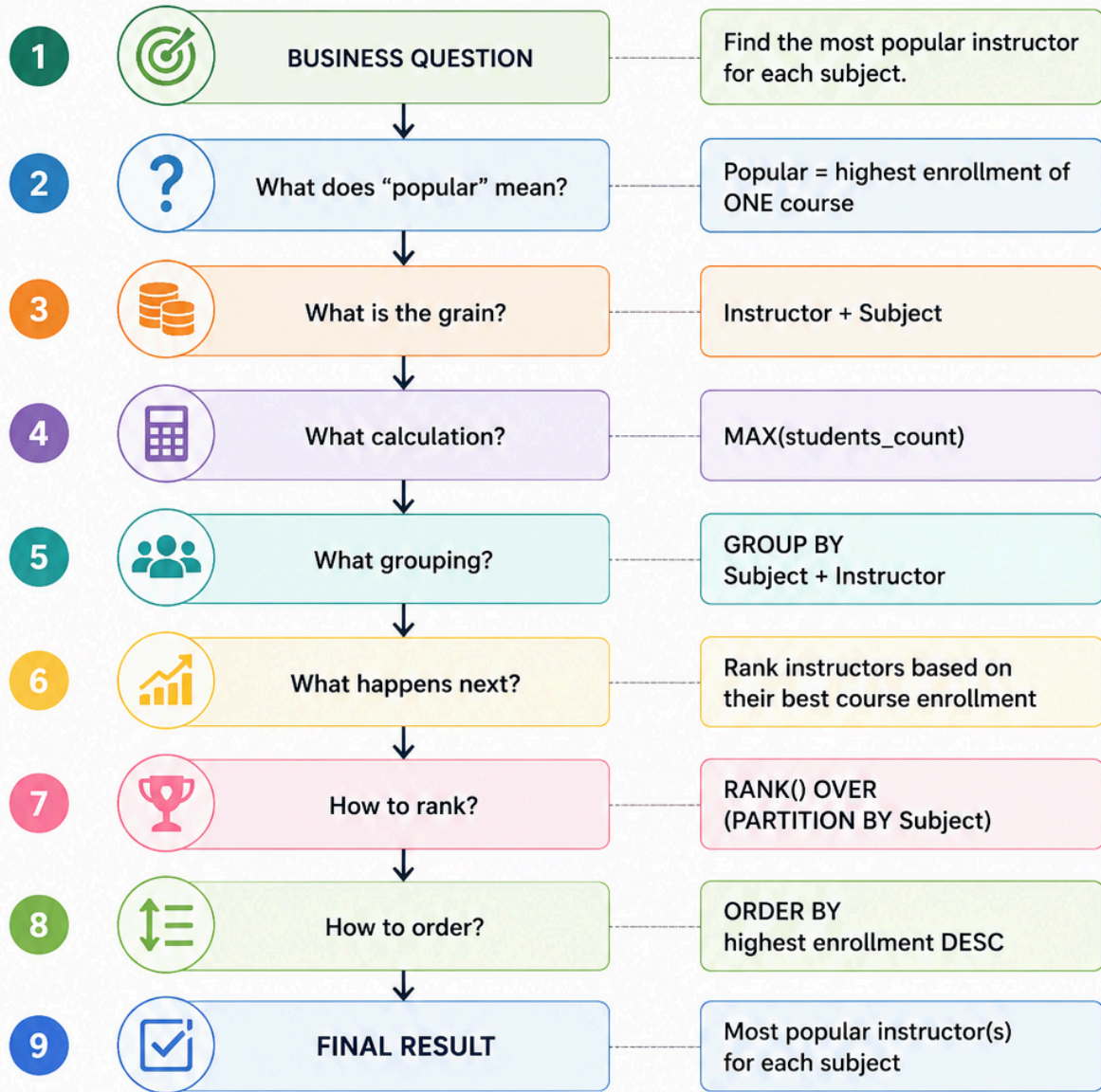
subject_name	instructor_name	max_course_enrollments	popularity_rank
Data Science	John Doe	500	1
Data Science	Sarah Lee	450	2
Data Science	Adam White	450	2
Web Dev	Alice Green	300	1
Web Dev	David Park	200	2

This exactly matches the expected output in the supplied problem.

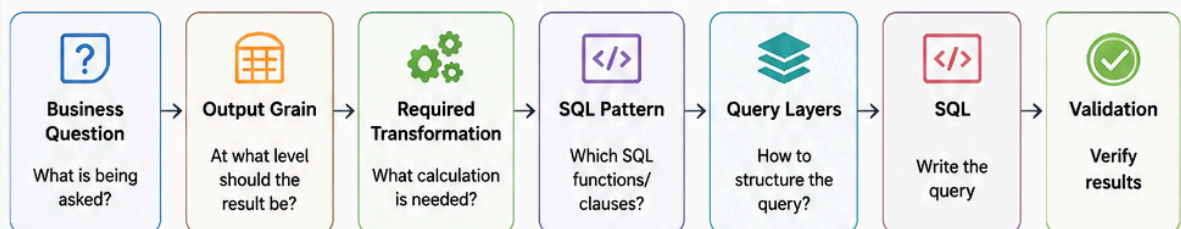
15. The real SQL problem-solving pattern

SQL PROBLEM-SOLVING PATTERN

Most Popular Instructor per Subject



MAPPING TO THE SQL FRAMEWORK



ONE-LINE MENTAL MODEL FOR TRAINEES



"First find each instructor's best course, then rank those **instructor-level bests** within each subject."